

inheritance

CSCI
134

sometimes we invest
considerable effort
into creating a new
class

```
class TreeNode:

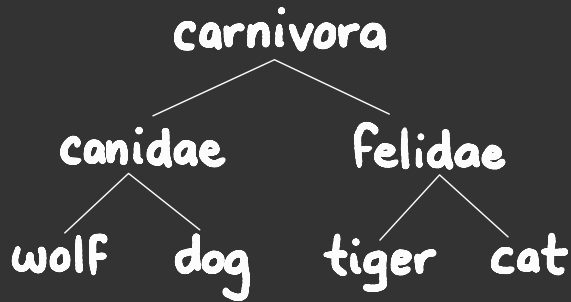
    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r

    def __contains__(self, lbl):
        return (self.label == lbl or
                (self.left != None and lbl in self.left) or
                (self.right != None and lbl in self.right))

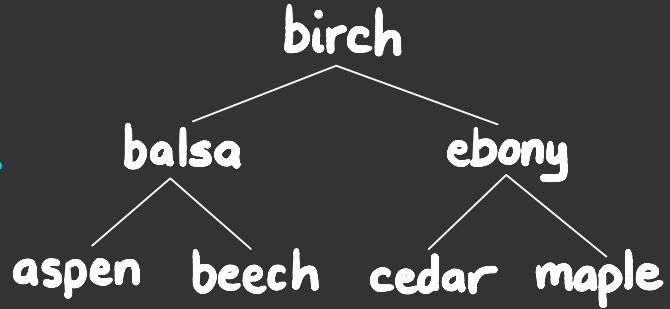
    def __len__(self):
        num_nodes = 1
        if self.left != None:
            num_nodes += len(self.left)
        if self.right != None:
            num_nodes += len(self.right)
        return num_nodes

    def __repr__(self):
        def repr_helper(node, indent):
            if node == None:
                return ""
            result = str(node.label) + "\n"
            if node.left != None:
                left_str = repr_helper(node.left, indent+2)
                result += (" " * indent) + "--L--> " + left_str
            if node.right != None:
                right_str = repr_helper(node.right, indent+2)
                result += (" " * indent) + "--R--> " + right_str
            return result
        return repr_helper(self, 0).strip()
```

and then realize there's
an important special case

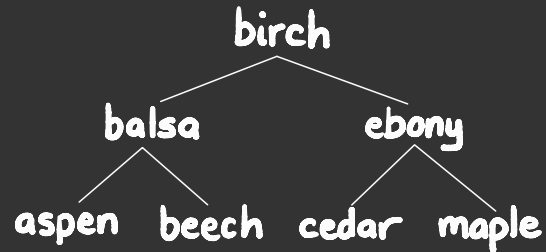
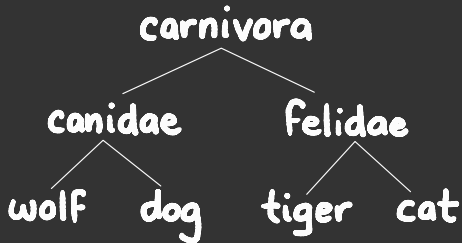


VS.



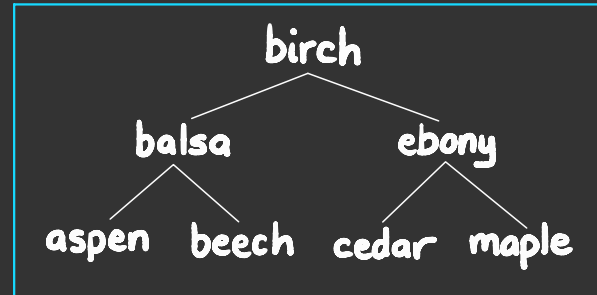
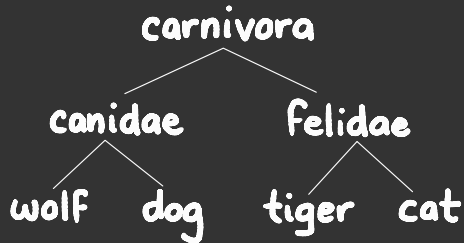
```
def __contains__(self, lbl):  
    return (self.label == lbl or  
            (self.left != None and lbl in self.left) or  
            (self.right != None and lbl in self.right))
```

this is a **good** way to
check whether **this tree**
contains a particular label

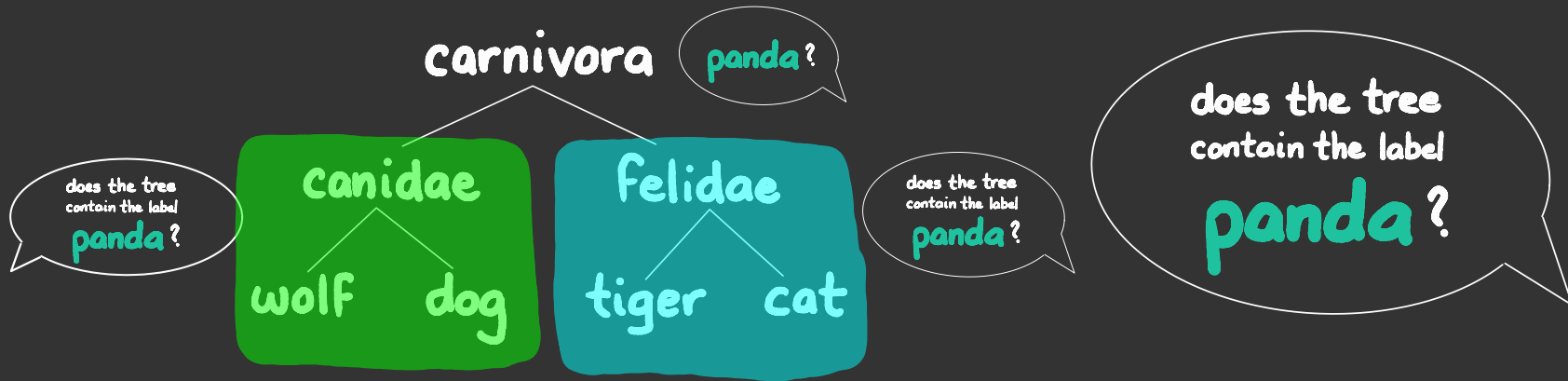



```
def __contains__(self, lbl):  
    return (self.label == lbl or  
            (self.left != None and lbl in self.left) or  
            (self.right != None and lbl in self.right))
```

but a **not-so-good** way
to check whether **this tree**
contains a particular label

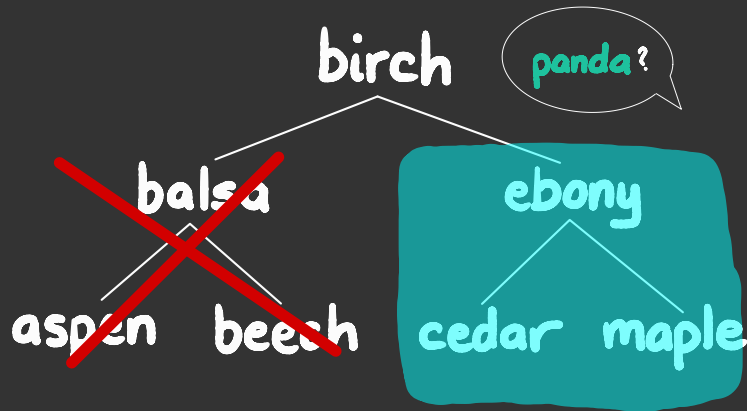


```
def __contains__(self, lbl):  
    return (self.label == lbl or  
            (self.left != None and lbl in self.left) or  
            (self.right != None and lbl in self.right))
```



because the labels aren't organized alphabetically,
we have to check **both** subtrees

```
def __contains__(self, lbl):  
    return (self.label == lbl or  
            (self.left != None and lbl in self.left) or  
            (self.right != None and lbl in self.right))
```



but this tree's labels **are** organized alphabetically,
so we **don't** have to check **both** subtrees

we could write another class
from scratch for this special case

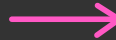
```
class TreeNode:

    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r

    def __contains__(self, lbl):
        return (self.label == lbl or
                (self.left != None and lbl in self.left) or
                (self.right != None and lbl in self.right))

    def __len__(self):
        num_nodes = 1
        if self.left != None:
            num_nodes += len(self.left)
        if self.right != None:
            num_nodes += len(self.right)
        return num_nodes

    def __repr__(self):
        def repr_helper(node, indent):
            if node == None:
                return ""
            result = str(node.label) + "\n"
            if node.left != None:
                left_str = repr_helper(node.left, indent+2)
                result += (" " * indent) + "--L--> " + left_str
            if node.right != None:
                right_str = repr_helper(node.right, indent+2)
                result += (" " * indent) + "--R--> " + right_str
            return result
        return repr_helper(self, 0).strip()
```



```
class TreeNode Binary SearchTreeNode

    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r

    def __contains__(self, lbl):
        return (self.label == lbl or
                (self.left != None and lbl in self.left) or
                (self.right != None and lbl in self.right))

    def __len__(self):
        num_nodes = 1
        if self.left != None:
            num_nodes += len(self.left)
        if self.right != None:
            num_nodes += len(self.right)
        return num_nodes

    def __repr__(self):
        def repr_helper(node, indent):
            if node == None:
                return ""
            result = str(node.label) + "\n"
            if node.left != None:
                left_str = repr_helper(node.left, indent+2)
                result += (" " * indent) + "--L--> " + left_str
            if node.right != None:
                right_str = repr_helper(node.right, indent+2)
                result += (" " * indent) + "--R--> " + right_str
            return result
        return repr_helper(self, 0).strip()
```

more efficient implementation

but most of the code is identical

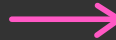
```
class TreeNode:

    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r

    def __contains__(self, lbl):
        return (self.label == lbl or
                (self.left != None and lbl in self.left) or
                (self.right != None and lbl in self.right))

    def __len__(self):
        num_nodes = 1
        if self.left != None:
            num_nodes += len(self.left)
        if self.right != None:
            num_nodes += len(self.right)
        return num_nodes

    def __repr__(self):
        def repr_helper(node, indent):
            if node == None:
                return ""
            result = str(node.label) + "\n"
            if node.left != None:
                left_str = repr_helper(node.left, indent+2)
                result += (" " * indent) + "--L--> " + left_str
            if node.right != None:
                right_str = repr_helper(node.right, indent+2)
                result += (" " * indent) + "--R--> " + right_str
            return result
        return repr_helper(self, 0).strip()
```



```
class Binary SearchTreeNode

    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r

    def __contains__(self, lbl):
        return (self.label == lbl or
                (self.left != None and lbl in self.left) or
                (self.right != None and lbl in self.right))

    def __len__(self):
        num_nodes = 1
        if self.left != None:
            num_nodes += len(self.left)
        if self.right != None:
            num_nodes += len(self.right)
        return num_nodes

    def __repr__(self):
        def repr_helper(node, indent):
            if node == None:
                return ""
            result = str(node.label) + "\n"
            if node.left != None:
                left_str = repr_helper(node.left, indent+2)
                result += (" " * indent) + "--L--> " + left_str
            if node.right != None:
                right_str = repr_helper(node.right, indent+2)
                result += (" " * indent) + "--R--> " + right_str
            return result
        return repr_helper(self, 0).strip()
```

Binary SearchTreeNode

more efficient
implementation

more efficient

maintaining two nearly identical classes can be arduous and error-prone

whenever
we update
this code

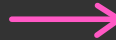
```
class TreeNode:

    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r

    def __contains__(self, lbl):
        return (self.label == lbl or
                (self.left != None and lbl in self.left) or
                (self.right != None and lbl in self.right))

    def __len__(self):
        num_nodes = 1
        if self.left != None:
            num_nodes += len(self.left)
        if self.right != None:
            num_nodes += len(self.right)
        return num_nodes

    def __repr__(self):
        def repr_helper(node, indent):
            if node == None:
                return ""
            result = str(node.label) + "\n"
            if node.left != None:
                left_str = repr_helper(node.left, indent+2)
                result += (" " * indent) + "--L--> " + left_str
            if node.right != None:
                right_str = repr_helper(node.right, indent+2)
                result += (" " * indent) + "--R--> " + right_str
            return result
        return repr_helper(self, 0).strip()
```



```
class BinarySearchTreeNode

    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r

    def __contains__(self, lbl):
        return (self.label == lbl or
                (self.left != None and lbl in self.left) or
                (self.right != None and lbl in self.right))

    def __len__(self):
        num_nodes = 1
        if self.left != None:
            num_nodes += len(self.left)
        if self.right != None:
            num_nodes += len(self.right)
        return num_nodes

    def __repr__(self):
        def repr_helper(node, indent):
            if node == None:
                return ""
            result = str(node.label) + "\n"
            if node.left != None:
                left_str = repr_helper(node.left, indent+2)
                result += (" " * indent) + "--L--> " + left_str
            if node.right != None:
                right_str = repr_helper(node.right, indent+2)
                result += (" " * indent) + "--R--> " + right_str
            return result
        return repr_helper(self, 0).strip()
```

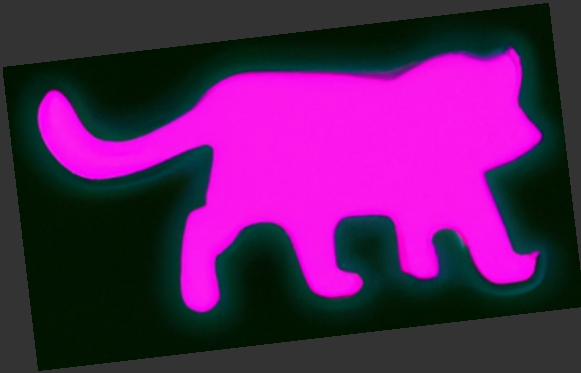
more efficient
implementation

more efficient

we must
remember
to update
this code

there's another way

time for



a game!

```
In [48]: zoo_game()  
YOU ARE AT A ZOO!
```

```
you spot an animal.  
do you examine it, or poke it? examine  
It is brown.  
what animal is it? otter  
it was an instance of <class 'zoo.Dog'>
```

```
In [49]: zoo_game()  
YOU ARE AT A ZOO!
```

```
you spot an animal.  
do you examine it, or poke it? poke  
wa-pa-pa-pa-pa-pa-pow  
what animal is it? fox  
it was an instance of <class 'zoo.Fox'>
```

```
In [50]: zoo_game()  
YOU ARE AT A ZOO!
```

```
you spot an animal.  
do you examine it, or poke it? poke  
you lost a finger.  
what animal is it? lion  
it was an instance of <class 'zoo.Tiger'>
```


classes can "inherit" the methods and attributes of a "parent" class

parent
constructor

```
class Animal:
    def __init__(self, sound, color):
        self.sound = sound
        self.color = color

    def poke(self):
        return 'It says "' + self.sound + '",'

    def describe(self):
        return 'It is ' + self.color + '.'
```

```
class Dog(Animal):
    def __init__(self, color):
        super().__init__("woof", color)
```

```
In [1]: from zoo import *

In [2]: scooby = Dog("brown")

In [3]: scooby.sound
Out[3]: 'woof'

In [4]: scooby.color
Out[4]: 'brown'

In [5]: scooby.poke()
Out[5]: 'It says "woof".'

In [6]: scooby.describe()
Out[6]: 'It is brown.'
```

iPython

classes can "override" methods inherited from a parent class

overridden
method



```
class Animal:
    def __init__(self, sound, color):
        self.sound = sound
        self.color = color

    def poke(self):
        return 'It says ' + self.sound + '.'
```

```
class Grizzly(Animal):
    def __init__(self):
        super().__init__("rawr", "brown")

    def poke(self):
        return 'You got mauled.'
```

```
In [2]: bear = Grizzly()
```

```
In [3]: bear.sound
```

```
Out[3]: 'rawr'
```

```
In [4]: bear.color
```

```
Out[4]: 'brown'
```

```
In [5]: bear.describe()
```

```
Out[5]: 'It is brown.'
```

```
In [6]: bear.poke()
```

```
Out[6]: 'You got mauled.'
```

iPython

but they can still call their
parent's version of these methods

```
class Animal:
    def __init__(self, sound, color):
        self.sound = sound
        self.color = color

    def poke(self):
        return 'It says "' + self.sound + '".'

    def describe(self):
        return 'It is ' + self.color + '.'
```

```
class Grizzly(Animal):
    def __init__(self):
        super().__init__("rawr", "brown")

    def describe(self):
        description = super().describe()
        return description[:-1] + ' and massive.'
```

`super()` refers to
the parent class

```
In [1]: from zoo import *
In [2]: bear = Grizzly()
In [3]: bear.describe()
Out[3]: 'It is brown and massive.'
```

iPython

```
class Animal:
    def __init__(self, sound, color):
        self.sound = sound
        self.color = color

    def poke(self):
        return 'It says "' + self.sound + '"'

    def describe(self):
        return 'It is ' + self.color + '.'
```

parent

```
class Fox(Animal):
    def __init__(self):
        super().__init__("wa-pa-pa-pa-pa-pow", "red")

    def describe(self):
        return super().describe() + " with a bushy tail"
```

```
class Dog(Animal):
    def __init__(self, color):
        super().__init__("woof", color)
```

```
class Cat(Animal):
    def __init__(self, color):
        super().__init__("meow", color)
```

parent

```
class Tiger(Cat):
    def __init__(self):
        super().__init__("orange")

    def poke(self):
        return "you lost a finger."
```

child classes can
be parents too